

Name:M S GOWRISH SHANKAR REDDY

Reg No:192424270

Sub date:10/8/26

Q1. Syntax-Directed Definition (SDD) for Expression Evaluation

Concept

The grammar uses **synthesized attributes**. Each non-terminal gets a .val attribute from its children.

For example:

$E \rightarrow E + T$

$E.val = E1.val + T.val$

The grammar also naturally handles precedence:

Parentheses $\rightarrow$ Multiplication/Division $\rightarrow$ Addition/Subtraction

C Program

```
#include <stdio.h>
```

```
#include <ctype.h>
```

```
const char *input;
```

```
int pos = 0;
```

```
int E();
```

```
int T();
```

```
int F();
```

```
/* E -> E + T | E - T | T */
```

```
int E()
```

```

{
    int value = T();

    while (input[pos] == '+' || input[pos] == '-')
    {
        char op = input[pos++];

        int right = T();

        if (op == '+')
            value = value + right;
        else
            value = value - right;
    }

    return value;
}

```

/* T -> T * F | T / F | F */

```

int T()
{
    int value = F();

    while (input[pos] == '*' || input[pos] == '/')
    {
        char op = input[pos++];

```

```

    int right = F();

    if (op == '*')
        value = value * right;
    else
        value = value / right;
}

return value;
}

/* F -> (E) | digit */
int F()
{
    int value;

    if (input[pos] == '(')
    {
        pos++;
        value = E();
        pos++; // Skip ')'
    }
    else
    {
        value = input[pos] - '0';
    }
}

```

```
        pos++;
    }

    return value;
}

int main()
{
    char expression[100];

    printf("Enter arithmetic expression: ");
    scanf("%99s", expression);

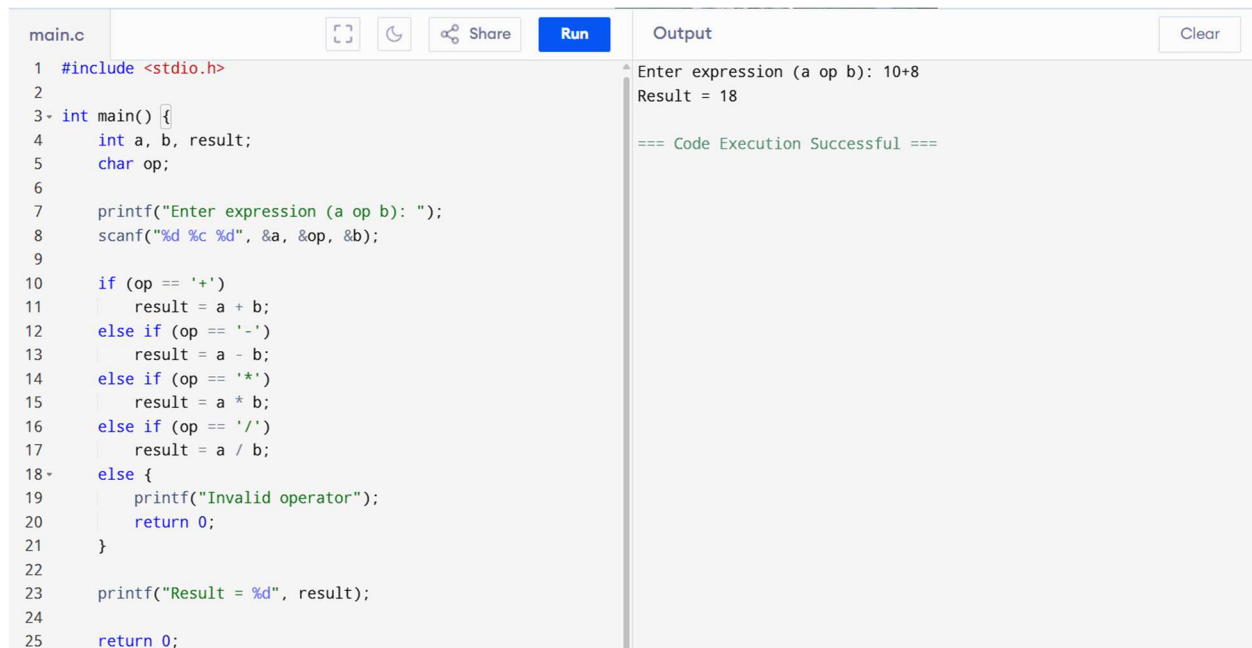
    input = expression;

    int result = E();

    printf("Final Result = %d\n", result);

    return 0;
}
```

Output:



```
main.c
1  #include <stdio.h>
2
3- int main() {
4      int a, b, result;
5      char op;
6
7      printf("Enter expression (a op b): ");
8      scanf("%d %c %d", &a, &op, &b);
9
10     if (op == '+')
11         result = a + b;
12     else if (op == '-')
13         result = a - b;
14     else if (op == '*')
15         result = a * b;
16     else if (op == '/')
17         result = a / b;
18- else {
19     printf("Invalid operator");
20     return 0;
21 }
22
23 printf("Result = %d", result);
24
25 return 0;
```

Output

Enter expression (a op b): 10+8
Result = 18

=== Code Execution Successful ===

Example

Enter arithmetic expression: 2+3*4

Final Result = 14

Here 3*4 is evaluated first because T handles * and / before E handles + and -.

SDD mapping

Production Semantic Rule

$E \rightarrow E + T$ $E.val = E1.val + T.val$

$E \rightarrow E - T$ $E.val = E1.val - T.val$

$E \rightarrow T$ $E.val = T.val$

$T \rightarrow T * F$ $T.val = T1.val * F.val$

$T \rightarrow T / F$ $T.val = T1.val / F.val$

$T \rightarrow F$ $T.val = F.val$

$F \rightarrow (E)$ $F.val = E.val$

Production Semantic Rule

$F \rightarrow \text{digit} \quad F.\text{val} = \text{digit}$

The .val attribute is **synthesized** because values move from child nodes toward the parent.

Q2. Postfix Evaluation Using S-Attributed Definition

Concept

A postfix expression such as:

$23*4+$

means:

$(2 * 3) + 4$

A **stack** is used for bottom-up evaluation.

For an S-attributed definition, attributes are synthesized from the symbols being processed.

C Program

```
#include <stdio.h>
```

```
#include <ctype.h>
```

```
#define MAX 100
```

```
int stack[MAX];
```

```
int top = -1;
```

```
void push(int value)
```

```
{
```

```
    stack[++top] = value;
```

```
}
```

```
int pop()
```

```
{
```

```
    return stack[top--];
```

```
}
```

```
int main()
```

```
{
```

```
    char postfix[MAX];
```

```
    printf("Enter postfix expression: ");
```

```
    scanf("%99s", postfix);
```

```
    for (int i = 0; postfix[i] != '\0'; i++)
```

```
    {
```

```
        char symbol = postfix[i];
```

```
        if (isdigit(symbol))
```

```
        {
```

```
            int value = symbol - '0';
```

```
            push(value);
```

```
            printf("Push %d\n", value);
```

```
        }
```

```
    else
```

```
{  
    int b = pop();  
    int a = pop();  
    int result;  
  
    switch (symbol)  
    {  
        case '+':  
            result = a + b;  
            break;  
  
        case '-':  
            result = a - b;  
            break;  
  
        case '*':  
            result = a * b;  
            break;  
  
        case '/':  
            result = a / b;  
            break;  
  
        default:  
            printf("Invalid operator\n");  
            return 1;  
    }  
}
```



```

    }

    printf("%d %c %d = %d\n", a, symbol, b, result);

    push(result);
}

}

printf("Final Result = %d\n", pop());

return 0;
}

```

Output:

main.c	Run	Output
<pre> 21- for (int i = 0; exp[i] != '\0'; i++) { 22- 23- if (isdigit(exp[i])) { 24- push(exp[i] - '0'); 25- } 26- else { 27- b = pop(); 28- a = pop(); 29- 30- if (exp[i] == '+') 31- result = a + b; 32- else if (exp[i] == '-') 33- result = a - b; 34- else if (exp[i] == '*') 35- result = a * b; 36- else if (exp[i] == '/') 37- result = a / b; 38- 39- push(result); 40- } 41- } 42- 43- printf("Result = %d", pop()); 44- 45- return 0; </pre>	Run	<pre> Enter postfix expression: 45*5+ Result = 25 === Code Execution Successful === </pre>

Example

Enter postfix expression: 23*4+

Push 2

Push 3

$2 * 3 = 6$

Push 4

$6 + 4 = 10$

Final Result = 10

Why is this S-attributed?

Every intermediate result is produced from already available values:

2 → synthesized value

3 → synthesized value

$2*3$ → synthesized value 6

4 → synthesized value

$6+4$ → synthesized value 10

So information flows **bottom-up**, which is characteristic of S-attributed definitions.

Q3. L-Attributed Definition Using Inherited Attributes

Concept

An **L-attributed definition** allows information to be passed from a parent or left sibling to a child.

Here, function parameters are used to simulate **inherited attributes**.

For:

$a + b * c$

Multiplication must be performed before addition.

We can represent the grammar as:

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid \epsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid \epsilon$

$F \rightarrow \text{operand}$

The accumulated value is passed as a function parameter.

C Program

```
#include <stdio.h>
```

```
int a, b, c;
```

```
/* F returns the value of an operand */
```

```
int F(char operand)
```

```
{
```

```
    if (operand == 'a')
```

```
        return a;
```

```

    if (operand == 'b')
        return b;

    if (operand == 'c')
        return c;

    return 0;
}

```

/*

T simulates:

$T \rightarrow F T'$

The result is passed to T' as an inherited attribute.

*/

```

int T(char *expr, int *pos)
{
    int value = F(expr[(*pos)++]);

    while (expr[*pos] == '*')
    {
        (*pos)++;

        int right = F(expr[(*pos)++]);
    }
}

```

```
    value = value * right;  
}
```

```
    return value;  
}
```

```
/*
```

E simulates:

$E \rightarrow T E'$

The value returned by T is passed forward
as an inherited/accumulated value.

```
*/
```

```
int E(char *expr)
```

```
{
```

```
    int pos = 0;
```

```
    int value = T(expr, &pos);
```

```
    while (expr[pos] == '+')
```

```
    {
```

```
        pos++;
```

```
        int right = T(expr, &pos);
```

```
        value = value + right;
    }

    return value;
}

int main()
{
    char expression[20];

    printf("Enter expression (example: a+b*c): ");
    scanf("%19s", expression);

    printf("Enter value of a: ");
    scanf("%d", &a);

    printf("Enter value of b: ");
    scanf("%d", &b);

    printf("Enter value of c: ");
    scanf("%d", &c);

    int result = E(expression);

    printf("Result = %d\n", result);
```

```
    return 0;
}
```

Output:

main.c	Output
<pre>1 #include <stdio.h> 2 3- int main() { 4 int a, b, c; 5 int result; 6 7 printf("Enter values of a, b and c: "); 8 scanf("%d %d %d", &a, &b, &c); 9 10 result = a + (b * c); 11 12 printf("Result = %d", result); 13 14 return 0; 15 }</pre>	<pre>Enter values of a, b and c: 1 2 3 Result = 7 === Code Execution Successful ===</pre>

Example

Enter expression (example: $a+b*c$): $a+b*c$

Enter value of a: 2

Enter value of b: 3

Enter value of c: 4

Result = 14

Because:

$b * c = 3 * 4 = 12$

$a + 12 = 2 + 12 = 14$

How inherited attributes are demonstrated

The computed value is passed into the next computation through function parameters.

For example:

```
int T(char *expr, int *pos)
```

and:

```
int E(char *expr)
```

The parameters act like **inherited information** passed down through the recursive/top-down evaluation process.

So:

Parent

↓ inherited information

Child

↓

Further computation

This demonstrates the basic idea of **L-attributed definitions**.

Q4. Type Expression Equivalence

Concept

A compiler needs to determine whether two type expressions represent the same type.

Examples:

`int == int`

`float == float`

`int != float`

`int* == int*`

`int* != float*`

The program below supports:

- `int`
- `float`
- `char`
- `int*`
- `float*`
- `char*`

C Program

```
#include <stdio.h>
```

```
#include <string.h>
```

```
typedef struct
```

```
{
```

```
    char baseType[20];
```

```
    int pointerLevel;
```

```
} Type;
```

```

Type parseType(char type[])
{
    Type t;
    t.pointerLevel = 0;

    int len = strlen(type);

    while (len > 0 && type[len - 1] == '*')
    {
        t.pointerLevel++;
        len--;
    }

    strncpy(t.baseType, type, len);
    t.baseType[len] = '\0';

    return t;
}

int areEquivalent(Type t1, Type t2)
{
    if (strcmp(t1.baseType, t2.baseType) != 0)
        return 0;

    if (t1.pointerLevel != t2.pointerLevel)
        return 0;

```

```
    return 1;
}

int main()
{
    char type1[50], type2[50];

    printf("Enter first type: ");
    scanf("%49s", type1);

    printf("Enter second type: ");
    scanf("%49s", type2);

    Type t1 = parseType(type1);
    Type t2 = parseType(type2);

    if (areEquivalent(t1, t2))
        printf("Equivalent\n");
    else
        printf("Not Equivalent\n");

    return 0;
}
```

Output:

main.c	Output
<pre>1 #include <stdio.h> 2 #include <string.h> 3 4 int main() { 5 char type1[20], type2[20]; 6 7 printf("Enter first type: "); 8 scanf("%s", type1); 9 10 printf("Enter second type: "); 11 scanf("%s", type2); 12 13 if (strcmp(type1, type2) == 0) 14 printf("Equivalent"); 15 else 16 printf("Not Equivalent"); 17 18 return 0; 19 }</pre>	<pre>Enter first type: int int Enter second type: Equivalent === Code Execution Successful ===</pre>

Examples

Enter first type: int

Enter second type: int

Equivalent

Enter first type: int

Enter second type: float

Not Equivalent

Enter first type: int*

Enter second type: int*

Equivalent

Enter first type: int*

Enter second type: float*

Not Equivalent

Logic

For two types to be equivalent:

Base types must match

AND

Pointer levels must match

For example:

int* → base = int, pointer = 1

int* → base = int, pointer = 1

Therefore:

Equivalent

But:

int* → int, 1

float* → float, 1

The base types differ, so:

Not Equivalent

Q5. Semantic Analyzer for Type Checking

Concept

The **semantic analyzer** checks whether an operation is valid according to the operand types.

For example:

int + int → Valid

int * float → Valid

int + char* → Type Error

float + int* → Type Error

Pointer arithmetic can have additional rules, but for this question we use a simple type-checking model.

C Program

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int isNumeric(char type[])
```

```
{
```

```
    return strcmp(type, "int") == 0 ||
```

```
        strcmp(type, "float") == 0 ||
```

```
        strcmp(type, "char") == 0;
```

```
}
```

```
int isPointer(char type[])
```

```
{
```

```
    int len = strlen(type);
```

```
    return len > 0 && type[len - 1] == '*';
```

```
}
```

```
int checkOperation(char type1[], char type2[], char op)
```

```
{
```

```
    /*
```

```
        Both operands are numeric.
```

```
        Allow +, -, *, /
```

```
    */
```

```
    if (isNumeric(type1) && isNumeric(type2))
```

```
    {
```

```
        if (op == '+' || op == '-' ||
```

```
            op == '*' || op == '/')
```

```
        {
```

```
            return 1;
```

```
        }
```

```
    }
```

```
    /*
```

```
        Pointer involved in arithmetic.
```

```
        This simple analyzer rejects these operations.
```

```
    */
```

```
    if (isPointer(type1) || isPointer(type2))
```

```
    {
```

```
        return 0;
```

```
    }
```

```
    return 0;
}

int main()
{
    char type1[20], type2[20];
    char op;

    printf("Enter first operand type: ");
    scanf("%19s", type1);

    printf("Enter operator (+, -, *, /): ");
    scanf(" %c", &op);

    printf("Enter second operand type: ");
    scanf("%19s", type2);

    printf("\nExpression: %s %c %s\n",
           type1, op, type2);

    if (checkOperation(type1, type2, op))
    {
        printf("Valid expression\n");
    }
    else
    {

```



```

        printf("Type error\n");
    }

    return 0;
}

```

Output:

main.c	Output
<pre> 3 4~ int main() { 5 char type1[20], type2[20], op; 6 7 printf("Enter first type: "); 8 scanf("%s", type1); 9 10 printf("Enter operator: "); 11 scanf(" %c", &op); 12 13 printf("Enter second type: "); 14 scanf("%s", type2); 15 16 if ((strcmp(type1, "int") == 0 17 strcmp(type1, "float") == 0) && 18 (strcmp(type2, "int") == 0 19~ strcmp(type2, "float") == 0)) { 20 21 printf("Valid expression"); 22 } 23~ else { 24 printf("Type error"); 25 } 26 27 return 0; </pre>	<pre> Enter first type: int Enter operator: - Enter second type: int Valid expression === Code Execution Successful === </pre>

Example 1

Enter first operand type: int

Enter operator (+, -, *, /): +

Enter second operand type: int

Expression: int + int

Valid expression

Example 2

Enter first operand type: int

Enter operator (+, -, *, /): *

Enter second operand type: float

Expression: int * float

Valid expression